

MonitorIoT

Panduan Deploy

Dokumen operasional final untuk dua repositori GitHub dan dua proyek Vercel yang sudah LIVE: kamus lengkap setiap parameter, environment variable, konfigurasi, dan kredensial beserta cara mendapatkannya, peta platform gratis Rp0 tanpa kartu kredit, prosedur deploy per komponen, gerbang verifikasi dan uji acceptance, manajemen rahasia, hingga sertifikasi produksi. Edisi 3: kendali darurat WAVE-7 (relay fail-safe, E-stop terindra, 12 ambang 3-lapis, gate ARM) + firmware v1.6.3 (audit noise inverter) + acceptance S6-S10.

Versi	3
Tanggal	2 September 2026
Objek	Next.js · PWA v2.0.0 · WAVE-7 · ESP32 v1.6.0/1.6.3
Status	PRODUCTION GRADE — LIVE · Rp0

DUA REPOSITORI GITHUB · DUA PROYEK VERCEL — SEMUA
PLATFORM GRATIS

Daftar Isi

1. Ringkasan Eksekutif	3
2. Arsitektur, Repositori, dan Deployment Aktif	4
2.3 Dua proyek Vercel: peran dan cara memakainya	6
2.4 Lapisan Kendali Darurat WAVE-7: Kontrak dan Cara Kerjanya	7
3. Kamus Parameter, Environment Variable, dan Kredensial	9
3.1 Peta parameter seluruh sistem	9
3.2 GAS - Script Properties (wajib diisi lewat editor)	10
3.3 GAS - konstanta PUSH_CONFIG (opsional, edit kode)	11
3.3b PLTS backend - kunci darurat di sheet Config (WAVE-7)	12
3.4 Aplikasi Next.js utama - environment variable Vercel dan panel Settings	13
3.5 PWA standalone - APP_CONFIG (js/config.js) dan sw.js	14
3.6 Firmware - konstanta sketch ESP32	15
3.7 Kredensial platform - cara mendapatkan, langkah demi langkah	17
4. Prosedur Deploy Komponen	18
4.1 Komponen GAS (backend push) - satu-satunya langkah yang tersisa	18
4.2 Menghubungkan aplikasi Next.js utama ke GAS	19
4.3 PWA standalone: injeksi konfigurasi dan hosting	19
4.4 Firmware ESP32	20
4.5 Aktivasi langganan push pertama	21
5. Verifikasi dan Uji Acceptance	21
5.1 Gerbang otomatis pra-go-live	21
5.2 Skenario acceptance manual (S1-S5)	23
5.3 Skenario acceptance darurat WAVE-7 (S6-S10)	23
6. Push Repositori dan Manajemen Rahasia	24
6.1 Klasifikasi rahasia vs nilai publik	24
6.2 Prosedur push pembaruan ke GitHub	25
6.3 Prosedur rotasi saat kebocoran	26
6.4 Cadangan lokal dan disiplin harian	27
7. Operasional Pasca Go-Live	27
7.1 Pemantauan rutin	27

7.2 Kuota GAS dan penyetelan kapasitas	27
7.3 Pemeliharaan berkala dan pembaruan	27
7.4 Operasional darurat WAVE-7 (ARM/DISARM dan ambang)	28
8. Penanganan Masalah	29
9. Sertifikasi Kesiapan Produksi	31
9.1 Bukti audit berlapis	31
9.2 Batas peran alat vs operator	32
9.3 Verdict	33
10. Lampiran A - Biaya Rp0: Platform Gratis dan Kuota	34
10.1 Peta platform yang dipakai dan kuotanya	34
10.2 MQTT: tidak dibutuhkan - dan itu sengaja	35
10.3 Batas yang benar-benar relevan dan perilakunya	36
10.4 Kompatibilitas jalur alternatif yang tetap gratis	36

1. Ringkasan Eksekutif

Panduan ini adalah dokumen operasional final untuk membawa sistem MonitorIoT ke produksi. Berbeda dari draf sebelumnya yang masih menggambarkan paket pra-rilis, seluruh jalur deployment kini SUDAH dieksekusi dan hidup: dua repositori GitHub berisi kode lengkap, dua proyek Vercel melayani HTTPS, dan aplikasi Next.js utama telah memiliki integrasi push-alarm natif - notifikasi alarm tetap tampil walau aplikasi ditutup, aksi notifikasi mengirim konfirmasi (ACK) ke backend, dan klik notifikasi membuka halaman Alarms melalui deep-link. Yang tersisa untuk operator hanyalah menghidupkan backend Google Apps Script (GAS) milik akun sendiri dan menempelkan dua nilai konfigurasi - kedua langkah itu dijelaskan tuntas pada Bab 3 dan 4.

Kualitas kode dijaga oleh rantai audit berlapis yang semuanya hijau pada saat dokumen ini disusun: 203 asersi regresi lintas komponen (35 krypto + 17 peramban + 151 kontrak silang K1-K8), 106 uji unit aplikasi Next.js, 46 asersi lapisan darurat GAS (EMERGENCY_*), 34 uji Python logika darurat firmware, 75 asersi kontrak GAS inti, 45 asersi kontrak wave-6, 83 uji Python logika firmware, nol galat tipe, dan nol galat lint. Sebuah bug decoder base64url kelas kritis ditemukan dan diperbaiki pada putaran integrasi terakhir: sekitar 93 persen kunci VAPID acak gagal didekode di peramban nyata karena normalisasi karakter berarah salah - perbaikannya dilintasi ke seluruh artefak (PWA vanilla, aplikasi Next.js, harness uji) dan dikunci dengan asersi regresi permanen.

Edisi 3 menghindari lapisan darurat fisik yang dikerahkan pada rilis WAVE-7: relay EMERGENCY master fail-safe (satu-satunya aktuator di seluruh tumpukan, arah hanya MEMUTUS) dengan E-stop fisik yang DIINDERAI firmware, kanal arus jenset ACS712 #2, diagram aliran energi animasi, 12 ambang pemicu yang divalidasi 3 lapis (PWA - GAS - firmware), plus empat perbaikan audit noise inverter pada firmware produksi v1.6.3 (WDT di dalam loop unduh OTA, recovery bus I2C terkunci, sensitivitas ACS712 di boot, WDT mengapit POST GAS). Seluruh kontrak dijelaskan di Bab 2.4; uji acceptance darurat S6-S10 ada di Bab 5.3. Deployment v3 tidak mengubah biaya platform: rantai tetap Rp0 tanpa kartu kredit.

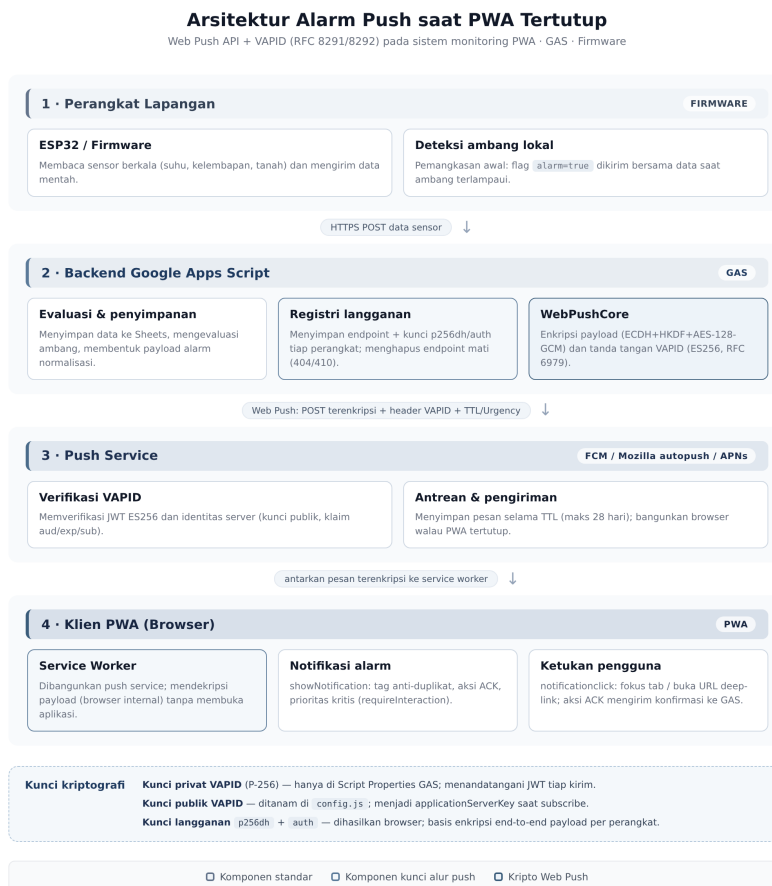
203+10 6 asersi regresi + uji unit, semua hijau	46+34+ 75 lapisan darurat GAS + firmware + kontrak inti	2+2 repo GitHub + proyek Vercel LIVE	Rp0 biaya seluruh platform - tanpa kartu kredit	1 aktuator darurat fail-safe (WAVE-7)
--	---	---	---	--

Seluruh platform yang dipakai sistem berjalan di paket gratis TANPA langganan dan TANPA input kartu kredit: Vercel paket Hobby (hosting dua proyek), Google Apps Script (backend push dan penyimpanan, akun Google gratis), push service peramban (FCM untuk Chrome/Android, APNs untuk Safari iOS 16.4+, Mozilla untuk Firefox - otomatis dan bebas biaya), dan GitHub untuk repositori. Lampiran A (Bab 10) memetakan batas kuota tiap platform terhadap kebutuhan instalasi maksimum dua HP dan satu modul monitoring, serta menjelaskan mengapa sistem ini sama sekali tidak membutuhkan MQTT.

Dokumen ini ditulis sebagai satu-satunya rujukan go-live. Bab 3 adalah kamus lengkap SETIAP parameter, variabel lingkungan, konfigurasi, dan kredensial - termasuk cara mendapatkan nilainya langkah demi langkah (membuat kunci VAPID, token perangkat, token GitHub, token Vercel, URL deployment GAS, hingga kredensial WiFi). Bab 4 memuat prosedur deploy per komponen dalam urutan ketergantungan yang benar. Bab 5 memuat gerbang verifikasi dan uji acceptance; Bab 6 menjelaskan manajemen rahasia dan rotasi; Bab 7 sampai 10 membahas operasional, penanganan masalah, sertifikasi, dan Lampiran A - peta platform gratis Rp0. Operator cukup membaca berurutan mulai Bab 3 sambil membuka dua akun: akun Google (untuk GAS) dan akun Vercel (untuk melihat deployment).

2. Arsitektur, Repositori, dan Deployment Aktif

Arsitektur MonitorIoT membagi tanggung jawab secara ketat. Firmware bertindak sebagai pemasok data: membaca sensor, mengevaluasi ambang secara lokal menjadi flag level, lalu mengirim laporan berkala ke endpoint ingest GAS dengan token perangkat. GAS memegang seluruh keputusan pengiriman: deteksi tepi naik per sensor memicu Web Push terenkripsi tepat satu kali per kejadian, tepi turun menutup kejadian dan mengirim satu notifikasi pulih. Sisi klien kini ada DUA pintu masuk yang sama-sama hidup: aplikasi Next.js utama (dasbor lengkap PLTS 48V dengan panel Settings terintegrasi) dan PWA standalone ringan (satu folder HTML statis). Keduanya memakai kontrak push yang identik, sehingga operator bebas memakai salah satu atau keduanya.



Gambar 2.1 - Arsitektur komponen MonitorIoT dan jalur komunikasinya (diagram dipakai ulang dari dokumen desain).

Seluruh kode produksi hidup di dua repositori GitHub milik akun `desvandi`. Repositori pertama memuat aplikasi Next.js utama (dasbor PLTS lengkap dengan seluruh gelombang pengembangan sebelumnya) plus folder `pwa-push-alarm` berisi PWA standalone; repositori kedua memuat folder `push-alarm` berisi backend GAS (Code.gs + WebPushCore.gs), firmware ESP32, alat deployment, dan suite regresi. Kedua repositori memakai struktur dokumentasi ramping: README tunggal yang komprehensif di akar + PDF panduan ini di akar (bukan terkubur di subfolder); dokumen desain `push-alarm` berada di folder `docs/` repositori firmware. Tabel 2.1 merinci isi dan status masing-masing; Tabel 2.2 merinci deployment Vercel yang saat ini aktif melayani.

Repositori GitHub (LIVE)	Isi utama	Status terakhir
github.com/desvandi/plts_monitor_PWA_only	Aplikasi Next.js utama "PLTS Monitor" (src/, package.json) + pwa-push-alarm/ PWA standalone + integrasi push-alarm natif (src/sw.ts, src/lib/push-alarm/, src/hooks/usePushAlarm.ts, panel Settings) + pustaka UI + uji vitest + README tunggal komprehensif + PDF panduan (akar repo)	main - struktur ramping: README komprehensif + panduan di akar
github.com/desvandi/plts_monitor_firmware-code.gs-etc	push-alarm/gas/Code.gs + WebPushCore.gs (backend push), push-alarm/firmware/MonitorIoT_Firmware.ino (ESP32), push-alarm/tools/ (6 alat deployment), push-alarm/tests/ (3 suite regresi + run-all.sh), repos.conf.example; ditambah code.gs/ (backend PLTS), firmware/ + firmware-generic/, scripts/ (suite uji), docs/ (wiring + desain push PDF); README tunggal komprehensif + PDF panduan (akar repo)	main - struktur ramping: README komprehensif + panduan di akar

Tabel 2.1 - Dua repositori GitHub produksi beserta isi dan statusnya.

Proyek Vercel (LIVE)	URL	Sumber dan pemicu deploy
jmse_plts_monitoring - aplikasi Next.js utama	https://jmsepltsmonitoring.vercel.app	Terhubung otomatis ke repo plts_monitor_PWA_only: setiap push ke branch main memicu build dan deploy ulang (auto-deploy)
plts-monitor-push-alarm - PWA standalone	https://plts-monitor-push-alarm.vercel.app	Dideploy dari folder pwa-push-alarm; melayani index.html, manifest.json, sw.js beserta header cache yang benar

Tabel 2.2 - Dua proyek Vercel aktif dan pemicu deploy-nya.

Prinsip yang dijaga di seluruh repositori: kode yang ter-commit SELALU bebas rahasia (templat placeholder atau konfigurasi runtime), nilai produksi ditanam hanya di tempat yang sah - Script Properties GAS, variabel lingkungan Vercel, memori firmware, atau isian panel Settings. Kontrak antar-komponen tetap acuan Tabel 11 dari laporan audit silang: K1 langganan, K2 penghapusan, K3 payload alarm, K4 ACK, K5 deep-link, K6 ambang alarm firmware-GAS, K7 kunci VAPID, dan K8 hardening produksi.

2.3 Dua proyek Vercel: peran dan cara memakainya

Dashboard Vercel menampilkan dua proyek pada tim yang sama - keduanya memang bagian dari sistem, tetapi perannya berbeda dan tidak saling bergantung. Keduanya berjalan gratis pada paket Hobby satu akun; memakai keduanya sekaligus tidak menambah biaya apa pun. Tabel berikut merinci perannya, kemudian dua daftar langkah menjelaskan cara memakai masing-masing.

Proyek	Peran dalam sistem	Status konfigurasi push
jmse_plts_monitoring (jmsepltsmonitoring.vercel.app)	APLIKASI UTAMA - dipakai harian. Dasbor PLTS 48V lengkap (sensor, grafik, alarms) + push alarm terintegrasi di panel Settings. Direkomendasikan untuk kedua HP.	Siap pakai: kunci VAPID sudah ter-bake otomatis dari env var build; URL GAS diisi runtime lewat panel Settings (Bab 4.2 jalur A) - tanpa redeploy.
plts-monitor-push-alarm (plts-monitor-push-alarm.vercel.app)	PWA STANDALONE CADANGAN - alternatif ringan khusus push alarm (hanya beberapa berkas statis; cocok untuk perangkat lambat). Opsional: boleh dipakai, diabaikan, atau dihapus.	Belum aktif: konfigurasinya build-time, masih placeholder - harus ditanam lewat apply-deploy-config.js lalu hosting ulang (Bab 4.3) sebelum bisa berlangganan push.

Tabel 2.3 - Perbandingan dua proyek Vercel dan kesiapan konfigurasinya.

- **Cara memakai aplikasi utama (jalur disarankan):** (1) buka <https://jmsepltsmonitoring.vercel.app> di browser/HP; (2) setelah backend GAS Anda ter-deploy (Bab 4.1), masuk menu Settings - panel "Server Push Alarm (GAS PushService)" - tempel URL /exec GAS pada kolom yang tersedia (kunci VAPID biasanya sudah terisi bawaan build) - Simpan; (3) ketuk "Aktifkan notifikasi" lalu izinkan notifikasi browser; (4) ketuk tombol uji push untuk memastikan jalur hidup. Setelah itu alarm tampil walau aplikasi ditutup; klik notifikasi membuka halaman Alarms; tombol "Tandai Ditangani" mengirim ACK.
- **Cara memakai PWA standalone (opsional):** jalankan injektor apply-deploy-config.js dengan URL GAS + kunci (Bab 4.3), hosting ulang build-nya (atau deploy ulang proyek Vercel dari folder build), lalu buka URL-nya dan ketuk "Aktifkan Notifikasi Alarm". Prosedur lengkap Bab 4.3.
- **ATURAN ANTI-DUPLIKAT - penting:** kedua aplikasi terhubung ke backend GAS yang sama. Satu perangkat cukup berlangganan push dari SATU aplikasi saja. Bila HP yang sama subscribe di kedua aplikasi, satu kejadian alarm menghasilkan DUA notifikasi (satu per langganan). Gunakan aplikasi utama untuk HP, dan biarkan PWA standalone sebagai cadangan.
- **Bolehkah menghapus salah satu?** Boleh dan aman: menghapus proyek plts-monitor-push-alarm di dashboard Vercel (Settings - Advanced - Delete) tidak memengaruhi aplikasi utama; kode sumbernya tetap tersimpan di folder pwa-push-alarm/ pada repo GitHub. Namun tidak ada kebutuhan menghapus - kuota paket Hobby tidak terganggu oleh proyek kecil yang jarang diakses (lihat Lampiran A).

2.4 Lapisan Kendali Darurat WAVE-7: Kontrak dan Cara Kerjanya

Rilis WAVE-7 (firmware-generic v1.6.0 + Code.gs + PWA v1.7, sudah LIVE) menambahkan satu-satunya aktuator fisik di seluruh sistem: RELAY DARURAT fail-safe - modul relai 5V optocoupler aktif-LOW yang menahan EMPAT kontaktor isolasi (PV, baterai, jenset, beban AC). Filosofinya kebalikan dari sistem kontrol biasa: relai BERENERGI = sistem RUN (GPIO LOW); relai LEPAS = TERISOLASI - dan SEMUA kegagalan rantai kontrol (boot, hang, crash-loop, brownout, trip, E-stop) berakhir pada relai lepas. Tombol E-stop fisik NC memutus jalur GND modul (murni hardware) SEKALIGUS diindera firmware lewat pin sense khusus - sehingga trip fisik tampil jelas di dasbor, bukan dideteksi dari hilangnya arus.

Semantik perintah sengaja dibedakan dari istilah umum dan identik di tiga lapis (PWA - GAS - firmware): ARM = operator meminta SISTEM MENYALA KEMBALI; DISARM = operator meminta ISOLASI (arah aman, SELALU diterapkan). Firmware boleh MENOLAK ARM dengan alasan bila pemicu masih aktif (dengan histeresis), masa pulih (recoverySec, default 60 detik) belum lewat, atau rantai crash beruntun terdeteksi - alasan penolakan dikirim balik ke operator dan tampil di dasbor. Perintah hanya hidup 10 menit di antrean (EMERGENCY_QUEUE_TTL_MIN): perintah basi berstatus EXPIRED dan tidak pernah diterapkan diam-diam.

Komponen WAVE-7	Isi	Tempat
EMERGENCY_COMMAND	ARM / DISARM / CONFIG dari operator - wajib ADMIN_TOKEN (fail-closed saat kosong)	Code.gs + PWA Kontrol Darurat
EMERGENCY_PENDING	perangkat mengambil perintah pending (piggyback TELEMETRI + poll khusus 15 detik)	firmware-generic v1.6.0
EMERGENCY_ACK	laporan hasil + alasan penolakan (terikat device_key, idempoten)	firmware-generic v1.6.0
EMERGENCY_EVENT	kejadian perangkat: TRIP/ESTOP/BOOT/CRA SH-LOOP/ARMED/DISARMED/REJECTED/...	Code.gs (whitelist tipe)
EMERGENCY_LOG	riwayat kejadian untuk dasbor	Code.gs + PWA
Sheet EmergencyQueue	antrean perintah (9 kolom) + TTL 10 menit + status PENDING/APPLIED/EXPIRED/REJECTED	Google Sheet
Sheet EmergencyEvents	log kejadian (7 kolom, rotasi 500 baris) + alert Telegram opsional (cooldown 2 menit)	Google Sheet
5 kolom telemetri v1.7	i_ac_gen, emg_state, emg_reason, emg_estop, emg_trips - di-append, baris lama = UNKNOWN	sheet Telemetry (migrasi lazy)
12 field CONFIG	5 pemicu sensor (VBAT rendah/tinggi + histeresis, arus DC/beban/jenset) + debounce + recovery + PIN GPIO - pin bisa dipindah TANPA reflash	EMERGENCY_COMMAND /CONFIG, divalidasi 3 lapis
Gate ARM firmware	pemicu aktif / masa pulih belum lewat / crash-chain - alasan dikirim balik ke operator	firmware-generic v1.6.0
Pins default	relai GPIO 27, sense E-stop GPIO 14 (opsional), ACS712 #2 jenset GPIO 32 - semuanya pindah-able via CONFIG	firmware-generic v1.6.0
Firmware produksi v1.6.3	perbaikan audit noise inverter: WDT di loop unduh OTA + stack otaTask 6K, recovery I2C 9x SCL, setSensitivity di boot (bug 1.85x), WDT mengapit POST GAS	firmware/ (modular, 2026-09-02)

Tabel 2.4 - Komponen kontrak WAVE-7 dan tempatnya.

URUTAN DEPLOY WAVE-7 (penting): (1) redeploy Code.gs WAVE-7 di GAS lebih dulu - kolom dan action baru bersifat aditif sehingga firmware lama tetap hidup tanpa perubahan; (2) bangun dan flash firmware-generic v1.6.0 bila memakai aktuator (binari dibangun operator lewat scripts/release_firmware_generic.py - toolchain tidak menyertai repositori); (3) PWA sudah ter-deploy otomatis di Vercel (menu Kontrol Darurat ikon perisai). Wiring 5 langkah, posisi E-stop, dan diagram lengkap: docs/wiring/emergency-relay.png di repo firmware + README bagian 6.5 dan 8 - dokumen rujukan tunggal untuk sisi hardware.

GPIO27 relai darurat aktif-LOW (default, pindah-able)	GPIO32 ACS712 #2 kanal jenset ke inverter	GPIO14 sense E-stop fisik (opsional, terlihat di dasbor)
---	---	--

3. Kamus Parameter, Environment Variable, dan Kredensial

Bab ini adalah kamus lengkap SETIAP nilai yang perlu Anda tentukan untuk menjalankan sistem - dikelompokkan per komponen dan ditutup dengan kamus kredensial platform. Aturan membacanya: kolom "Cara mendapatkan" selalu memuat langkah persis atau perintah persis; jangan mengarang nilai. Sebagian besar nilai sekali-setel-seumur instalasi; bagian rotasi (Bab 6.3) menjelaskan cara menggantinya bila diperlukan. Nilai produksi untuk instalasi ini SUDAH dibuat dan tersimpan lokal pada folder rahasia deploy/ (vapid-keys.json, device-token.txt, script-properties.txt) - Bab 3.1 memuat lokasi persis tiap nilai, dan Bab 6.4 aturan penyimpanannya.

3.1 Peta parameter seluruh sistem

Tabel berikut adalah peta induk: baris mana pun dapat dilacak ke subbab rinciannya. Perhatikan bahwa SATU nilai sering dipakai di beberapa tempat sekaligus - kunci VAPID publik yang sama harus hadir identik di empat tempat, URL GAS yang sama di tiga tempat. Kolom "sumber nilai" menunjuk dari mana nilai itu berasal sebelum ditempel.

Parameter	Tempat nilai ditempel	Sumber nilai (cara mendapatkan)
VAPID_PUBLIC_KEY (kunci publik, 87 karakter base64url)	1) Script Properties GAS 2) variabel lingkungan Vercel NEXT_PUBLIC_USH_VAPID_PUBLIC_KEY 3) panel Settings aplikasi Next.js 4) js/config.js + sw.js PWA standalone	Perintah: node tools/generate-vapid-keys.js - salin baris Public Key. Nilai produksi siap pakai: deploy/vapid-keys.json (kunci "publicKey"). Detail Bab 3.2.
VAPID_PRIVATE_KEY (RAHASIA, 43 karakter)	HANYA Script Properties GAS	Perintah yang sama, baris Private Key; disimpan di deploy/vapid-keys.json (kunci "privateKey"). Tidak pernah ke klien/git/log. Detail Bab 3.2.
URL Web App GAS (API_BASE, berakhiran /exec)	1) panel Settings aplikasi Next.js (atau env var NEXT_PUBLIC_PUSH_API_BASE) 2) js/config.js + sw.js PWA 3) GAS_URL firmware	Dihasilkan saat menekan Deploy di editor GAS (Bab 4.1 langkah 5). Format <a href="https://script.google.com/macros/s/<ID>/exec">https://script.google.com/macros/s/<ID>/exec .
FW_DEVICE_TOKEN (RAHASIA, 43 karakter)	1) Script Properties GAS 2) konstanta DEVICE_TOKEN firmware yang di-flash	Perintah: node tools/prep-deploy-secrets.js - token acak 32-byte base64url. Nilai produksi: deploy/device-token.txt. Alternatif: openssl rand -base64 32. Detail Bab 3.2 dan 3.6.
VAPID_SUBJECT (kontak pengirim)	Script Properties GAS (opsional)	Alamat surel Anda, format mailto:nama@domain. Detail Bab 3.2.
Kredensial WiFi (SSID + password)	Konstanta WIFI_SSID/WIFI_PASS firmware	Halaman admin router Anda; WAJIB jaringan 2,4 GHz. Detail Bab 3.6.
Token GitHub (untuk push)	URL remote git / manajer kredensial (tidak pernah di berkas)	Dibuat di github.com - langkah persis di Bab 3.7.
Token Vercel (untuk CLI/API deploy)	Variabel lingkungan mesin operator / kredensial CLI	Dibuat di vercel.com/account/tokens - langkah persis di Bab 3.7.

Tabel 3.1 - Peta induk seluruh parameter: tempat ditempel dan sumber nilainya.

3.2 GAS - Script Properties (wajib diisi lewat editor)

Script Properties adalah "environment variable"-nya Google Apps Script: diisi sekali lewat editor tanpa menyentuh kode. Cara membuka formulirnya: buka script.google.com - pilih proyek - ikon Project Settings (roda gigi) - bagian Script Properties - tombol Add script property; masukkan nama properti pada kolom Property dan nilainya pada kolom Value. Berkas lokal `deploy/script-properties.txt` berisi ketiga baris wajib dalam format siap salin-tempel (nama TAB nilai). Tiga properti pertama wajib; satu alternatif untuk armada perangkat; dua sisanya opsional.

Properti	Wajib	Nilai / contoh	Cara mendapatkan / menentukan
VAPID_PUBLIC_KEY	Ya	87 karakter base64url, diawali B, decode 65 byte berawalan 04	Salin dari deploy/script-properties.txt (juga ada di deploy/vapid-keys.json kunci publicKey). Untuk membuat dari nol: node tools/generate-vapid-keys.js --save - baca baris Public Key. WAJIB identik dengan nilai di Vercel/panel Settings/PWA.
VAPID_PRIVATE_KEY	Ya	43 karakter base64url (32 byte)	Salin dari deploy/script-properties.txt (atau kunci privateKey vapid-keys.json). RAHASIA - hanya hidup di Script Properties dan folder rahasia lokal berizin 600; tidak pernah di klien, git, atau log.
FW_DEVICE_TOKEN	Ya	43 karakter base64url acak	Salin dari deploy/script-properties.txt (juga deploy/device-token.txt). Membuat dari nol: node tools/prep-deploy-secrets.js --secrets-dir deploy (idempoten), atau openssl rand -base64 32. Harus sama persis dengan DEVICE_TOKEN firmware.
FW_DEVICE_TOKENS	Alternatif	[{"deviceId":"esp32-01","token":"..."}]	Untuk lebih dari satu perangkat: susun JSON array pasangan id+token sendiri (token per perangkat dari generator yang sama). Bila properti ini ada, FW_DEVICE_TOKEN diabaikan dan pasangan dicocokkan ketat (fail-closed).
VAPID_SUBJECT	Tidak	mailto:ops@domain-anda.id	Alamat surel yang bisa dihubungi push service bila ada masalah pengiriman (ketentuan RFC 8292). Tulis mailto: lalu alamat surel Anda sendiri. Menimpa konstanta SUBJECT tanpa edit kode.
TEST_PUSH_MIN_INTERVAL_MS	Tidak	60000	Jeda minimal antar permintaan testPush publik (anti-spam). 0 = tanpa batas. Biarkan 60000 di produksi; turunkan hanya saat uji meja.

Tabel 3.2 - Script Properties GAS: wajib, alternatif, dan opsional.

3.3 GAS - konstanta PUSH_CONFIG (opsional, edit kode)

Konstanta di header Code.gs jarang perlu diubah. Dua nilai pertama (SUBJECT dan TEST_PUSH_MIN_INTERVAL_MS) dapat ditimpa lewat Script Properties sehingga tidak perlu menyentuh kode sama sekali; sisanya adalah perilaku inti yang sudah dikalibrasi terhadap batas kuota dan ukuran payload Web Push. Tabel ini cukup dibaca sekali untuk memahami batas-batas sistem.

Konstanta	Default	Fungsi dan kapan mengubah
SUBJECT	mailto:admin@monitor-iot.contoh.id	Kontak pengirim VAPID (fallback bila properti VAPID_SUBJECT kosong). Tetapkan lewat properti, bukan edit kode.
TEST_PUSH_MIN_INTERVAL_MS	60000	Rate-limit testPush (fallback properti). Lihat Tabel 3.2.
TTL	86400	Umur pesan di push service (detik, maks 28 hari). 24 jam agar alarm dicoba ulang saat perangkat pelanggan offline semalaman.
BATCH_SIZE	100	Maksimum langganan per eksekusi pengiriman; menjaga kuota UrlFetchApp dan batas eksekusi 6 menit. Turunkan bila muncul timeout pada instalasi besar.
MAX_PAYLOAD_CHARS	3000	Batas aman ukuran JSON payload sebelum dipotong otomatis; push service umumnya menolak lebih dari 4096 byte.
USE_SHEET_STORAGE	true	true = langganan disimpan di sheet push_subscriptions (mudah diaudit); false = Script Properties (cukup untuk di bawah 100 perangkat, tanpa Spreadsheet terikat).
SHEET_NAME	push_subscriptions	Nama sheet registri langganan; sheet dibuat otomatis saat langganan pertama bila USE_SHEET_STORAGE aktif.
PRUNE_AFTER_DAYS	90	Langganan tak terlihat lebih lama dari ini dibuang otomatis.
NOTIFY_RESOLVE	true	Kirim satu notifikasi PULIH saat alarm kembali normal.
MAX_ALARM_LOG	200	Ukuran buffer melingkar jejak kejadian ALARM_LOG.
MAX_SENSORS_PER_REPORT	24	Batas sensor per laporan firmware (klien merender maks 24).
THRESHOLDS	suhu>40 kritis, udara<25, tanah<30	Jaring pengaman sisi GAS: hanya dipakai bila laporan firmware TIDAK menyertakan flag alarm. Samakan dengan ambang firmware bila mengubah.

Tabel 3.3 - Konstanta PUSH_CONFIG di Code.gs dan implikasinya.

3.3b PLTS backend - kunci darurat di sheet Config (WAVE-7)

Backend PLTS (Code.gs pada Master Sheet) membaca konfigurasi dari tab Config. WAVE-7 menambahkan tiga kunci perilaku di sana - seluruhnya OPSIONAL: deployment lama tanpa kunci ini mendapat nilai bawaan yang sama lewat fallback internal, tanpa langkah migrasi. AMBANG PEMICU berbeda: tidak disimpan sebagai kunci Config, melainkan DIKIRIM sebagai perintah CONFIG per perangkat (12 field, divalidasi 3 lapis dan dipersisten di LittleFS perangkat) - Tabel 2.4. Kunci ADMIN_TOKEN adalah gerbang aksi operator: selama kosong, ARM/DISARM ditolak (fail-closed) - isi dengan nilai rahasia kuat; PWA menyimpan salinannya per perangkat di localStorage (halaman /setup, kolom Admin Token) - tidak pernah masuk repositori atau build.

Kunci Config / field	Default	Fungsi
ADMIN_TOKEN	(kosong)	Gerbang EMERGENCY_COMMAND (ARM/DISARM/CONFIG); sama kelasnya dengan gate OTA_PUBLISH; fail-closed selama kosong
EMERGENCY_QUEUE_TTL_MIN	10	Umur perintah di EmergencyQueue; basi = EXPIRED (tidak pernah diterapkan diam-diam)
EMERGENCY_EVENTS_MAX_ROWS	500	Rotasi sheet EmergencyEvents (log kejadian perangkat)
EMERGENCY_ALERT_COOLDOWN_MIN	2	Jeda minimum antar alert Telegram per topik kejadian
CONFIG: vbatLowV / vbatLowHystV	42.0 / 1.0	Pemicu tegangan rendah + histeresis pulih (V; rentang 30-60 / 0.1-5)
CONFIG: vbatHighV / vbatHighHystV	55.0 / 1.0	Pemicu tegangan tinggi + histeresis (V; rentang 48-60 / 0.1-5)
CONFIG: iDcOverA	110.0	Pemicu arus DC berlebih (A; rentang 10-120)
CONFIG: iAcLoadOverA / iAcGenOverA	28.0 / 28.0	Pemicu arus AC beban / jenset (A; rentang 5-40)
CONFIG: debounceN	3	Jumlah siklus berturut-turut sebelum trip (rentang 1-10)
CONFIG: recoverySec	60	Masa pulih minimum sebelum ARM diterima (detik; 0-3600)
CONFIG: relayPin / estopPin / estopEnabled	27 / 14 / 1	Pin GPIO relai + sense E-stop - dipindah TANPA reflash (12-39 / -1..39 / 0-1)

Tabel 3.3b - Kunci dan field konfigurasi darurat WAVE-7.

3.4 Aplikasi Next.js utama - environment variable Vercel dan panel Settings

Aplikasi Next.js utama (jmsepltsmonitoring.vercel.app) membaca konfigurasi push dari DUA lapis. Lapis pertama adalah nilai bawaan build-time lewat variabel lingkungan Vercel - ditanam saat build, sehingga pengguna baru melihat konfigurasi terisi otomatis (zero-touch). Lapis kedua adalah isian runtime lewat panel Settings di dalam aplikasi - disimpan di penyimpanan peramban (IndexedDB untuk service worker, localStorage untuk UI) dan menimpa nilai bawaan. Bila kedua lapis kosong, fitur push menampilkan status "belum dikonfigurasi" dan tombol berpandu ke panel.

Cara menyetel variabel lingkungan di Vercel: buka vercel.com - login - pilih proyek jmse_plts_monitoring - tab Settings - kiri Environment Variables - isi kolom Key, Value, centang ketiga lingkungan (Production, Preview, Development) - Add - LALU WAJIB Redeploy (menu Deployments - deploy terbaru - menu tiga titik - Redeploy) karena nilai NEXT_PUBLIC_* di-bake ke bundel saat build; menambah variabel tanpa redeploy tidak mengubah apa pun. Nilai yang ditempel adalah nilai PLAIN (string base64url apa adanya),

bukan JSON.

Variabel lingkungan	Wajib	Nilai	Cara mendapatkan / menentukan
NEXT_PUBLIC_PUSH_VAPID_PUBLIC_KEY	Ya (disarankan)	87 karakter base64url, diawali B	deploy/vapid-keys.json kunci publicKey - tempel APA ADANYA (satu baris, tanpa tanda kutip). HARUS identik dengan VAPID_PUBLIC_KEY di Script Properties GAS (kontrak K7). Sudah terpasang pada proyek Vercel saat dokumen ini disusun; verifikasi: Bab 5.1 baris terakhir.
NEXT_PUBLIC_PUSH_API_BASE	Opsional	URL /exec GAS atau kosong	URL deployment Web App GAS (Bab 4.1 langkah 5). Bila dikosongkan atau dilewatkan, operator mengisi URL lewat panel Settings saat pertama memakai fitur push - sama sahnyanya. Menyetelnya lewat env var menghilangkan satu langkah manual per peramban.

Tabel 3.4 - Variabel lingkungan Vercel aplikasi Next.js utama.

Lapis kedua - panel Settings runtime: buka aplikasi - menu Settings - panel "Server Push Alarm (GAS PushService)" - isi dua kolom: "URL Web App GAS (.../exec)" dan "Kunci Publik VAPID (base64url)" - Simpan. Nilai tersimpan lokal di peramban itu dan tersinkron ke service worker otomatis pada muat ulang berikutnya. Gunakan jalur ini bila Anda ingin menguji URL deployment berbeda tanpa redeploy, atau bila env var sengaja tidak disetel. Panel juga menyediakan tombol uji push (dibatasi 60 detik oleh GAS) dan menampilkan status langganan peramban saat ini.

3.5 PWA standalone - APP_CONFIG (js/config.js) dan sw.js

PWA standalone (pwa-push-alarm/) menanam konfigurasi di dua berkas: js/config.js untuk aplikasi dan konstanta API_BASE di baris atas sw.js untuk service worker (SW tidak dapat membaca config.js). Kedua nilai API_BASE HARUS identik - gerbang verifikasi menolak build yang tidak konsisten. Cara pengisian yang benar adalah lewat injektor satu perintah (Bab 4.4) yang menanam keduanya sekaligus, bukan edit manual dua berkas. Konfigurasi ini TIDAK berlaku untuk aplikasi Next.js utama (yang memakai jalur env var/panel Settings Bab 3.4).

Parameter	Default	Cara mendapatkan / menentukan
API_BASE (config.js)	placeholder GANTI_DENGAN...	URL deployment Web App GAS (berakhiran /exec), didapat setelah Deploy di editor GAS. Ditanam injektor --url.
API_BASE (sw.js)	placeholder (harus = config.js)	Otomatis sama dengan config.js bila diisi injektor; diverifikasi oleh tools/verify-deployment.js.
VAPID_PUBLIC_KEY	placeholder GANTI_DENGAN...	Kunci publik VAPID (sama dengan Script Properties GAS dan env var Vercel). Ditanam injektor dari deploy/vapid-keys.json. Aman ditanam di klien.
APP_VERSION	2.0.0	Diagnostik dan cache busting; naikan setiap perubahan perilaku PWA/SW.
POLL_INTERVAL_MS	30000	Interval polling snapshot saat aplikasi terbuka; polling berhenti otomatis saat tab disembunyikan.
FETCH_TIMEOUT_MS	15000	Batas waktu tiap permintaan ke GAS (Web App bisa lambat saat dingin).
CONNECTION_BANNER_AFTER_FAILURES	2	Jumlah kegagalan beruntun sebelum banner status koneksi tampil.

Tabel 3.5 - Konfigurasi PWA standalone dan sumber nilainya.

3.6 Firmware - konstanta sketch ESP32

Seluruh konfigurasi firmware berada di blok KONFIGURASI pada push-alarm/firmware/MonitorIoT_Firmware/MonitorIoT_Firmware.ino. Empat nilai pertama wajib diisi sebelum flash; sisanya adalah penyetelan perilaku. Cara pengisian yang disarankan lewat injektor (--url, --token, --ssid, --pass, --device-id) sehingga GAS_URL dan DEVICE_TOKEN dijamin sinkron dengan sisi GAS; nilai ambang TEMP_MAX/SOIL_MIN/HUM_MIN harus disamakan manual dengan THRESHOLDS GAS bila diubah.

Konstanta	Default	Cara mendapatkan / menentukan
WIFI_SSID / WIFI_PASS	GANTI_NAMA_WIFI / GANTI_PASSWORD_WIFI	Kredensial WiFi lokasi perangkat: buka halaman admin router (umumnya 192.168.0.1 atau 192.168.1.1), lihat nama SSID dan kata sandi. WAJIB 2,4 GHz (ESP32 tidak mendukung 5 GHz). Isi lewat injektor --ssid/--pass atau langsung di IDE.
GAS_URL	placeholder (harus = API_BASE PWA)	URL deployment GAS yang sama dengan sisi klien; ditanam injektor --url.
DEVICE_ID	esp32-greenhouse-01	Nama unik perangkat; harus cocok dengan deviceId pada mode FW_DEVICE_TOKENS. Bebas untuk mode token tunggal.
DEVICE_TOKEN	GANTI_TOKEN_PERANGKAT	Sama persis dengan FW_DEVICE_TOKEN GAS. Sumber: deploy/device-token.txt. Ditanam injektor. JANGAN di-commit dalam bentuk terisi.
FW_VERSION	1.1.0	Tunggal-sumber (#define); naikan setiap perubahan perilaku agar jejak laporan di GAS dapat dibedakan.
SENSOR_MODE_*	SIMULATED aktif	Pilih SATU: SIMULATED untuk uji meja tanpa perangkat, DHT22 untuk sensor nyata (butuh pustaka Adafruit DHT + Unified Sensor).
REPORT_INTERVAL_MS	60000	Periode laporan rutin. 60 detik = 1.440 laporan/hari, jauh di bawah batas UrlFetchApp.
HTTP_TIMEOUT_MS	15000	Batas waktu POST ke GAS per percobaan.
WIFI_TIMEOUT_MS	20000	Batas waktu join WiFi saat boot.
MAX_BACKOFF_MS	480000	Plafon backoff eksponensial saat kirim gagal (maks 8 menit).
NTP_TIMEOUT_MS	10000	Batas waktu sinkronisasi waktu; gagal - fallback millis (nilai reportedAt relatif, hanya diagnostik).
TEMP_MAX / HUM_MIN / SOIL_MIN	40.0 / 25.0 / 30.0	Ambang alarm lokal (C, persen, persen). Samakan dengan THRESHOLDS GAS. Naikkan SIM_TEMP_CENTER melewati ambang untuk memicu alarm uji.
SIM_*_CENTER	29.5 / 71.0 / 55.0	Pusat random-walk mode simulasi.
DHT_PIN / SOIL_PIN / BUZZER_PIN	4 / 34 / -1	Pin GPIO sensor dan buzzer; -1 mematikan buzzer. Kalibrasi ADC tanah: kering sekitar 3200, basah sekitar 1300.
TLS_SKIP_CERT_VERIFY	tidak didefinisikan (verifikasi AKTIF)	Aktifkan HANYA di jaringan ber-intersepsi TLS (proxy/hotspot captive) - koneksi menjadi rentan MITM. Default produksi: mati.

Tabel 3.6 - Konstanta firmware dan cara menentukannya.

3.7 Kredensial platform - cara mendapatkan, langkah demi langkah

Empat kredensial/akun berikut dibutuhkan HANYA pada tahap mengelola (men-deploy GAS, mendorong pembaruan kode, mengelola proyek Vercel) - pengguna akhir aplikasi tidak membutuhkan satu pun dari mereka. Semua token diperlakukan seperti kata sandi: simpan di manajer kredensial, jangan pernah di berkas repo atau repos.conf, dan rotasi segera bila tercuri (Bab 6.3). Seluruh akun dibuat dan dipakai gratis - tidak ada satu pun yang meminta kartu kredit (pembuktian kuota dan biaya ada di Lampiran A).

Kredensial	Dipakai untuk	Cara mendapatkan (klik demi klik)
Akun Google (gratis)	Menampung proyek Apps Script backend push	1) Buka accounts.google.com/signup bila belum punya. 2) Login lalu buka script.google.com - akun siap dipakai (Bab 4.1). Tidak perlu akun berbayar; kuota gratis cukup untuk sistem ini.
Token akses personal GitHub (PAT)	git push pembaruan kode ke 2 repositori lewat HTTPS	1) Login github.com dengan akun pemilik repo. 2) Klik foto profil kanan-atas - Settings. 3) Gulir ke bawah, klik Developer settings (menu kiri). 4) Personal access tokens - Tokens (classic). 5) Generate new token (classic) - beri nama, masa berlaku, centang scope "repo" penuh. 6) Generate - SALIN sekaligus (berawalan ghp_, hanya sekali tampil). 7) Pakai di URL push: <code>https://<TOKEN>@github.com/desvandi/<repo>.git</code> - hapus dari URL setelah push (Bab 6.2).
Token Vercel	Menambah/mengubah variabel lingkungan dan deploy lewat CLI/API	1) Login vercel.com . 2) Klik avatar kanan-atas - Account Settings. 3) Menu Tokens. 4) Create - beri nama, scope tim, masa berlaku. 5) SALIN (berawalan vcp_...). 6) Ekspor di terminal: <code>export VERCEL_TOKEN=...</code> lalu pakai pada perintah CLI atau simpan aman. Catatan: mengelola lewat dashboard vercel.com (login biasa) tidak membutuhkan token sama sekali.
Kredensial WiFi lokasi	Menghubungkan ESP32 ke internet	Buka halaman admin router (192.168.0.1 / 192.168.1.1 / 192.168.8.1 sesuai merek) - cari nama jaringan (SSID) dan kata sandi WPA2. Pastikan jaringan 2,4 GHz aktif.

Tabel 3.7 - Kredensial platform: kegunaan dan langkah mendapatkannya.

Catatan penting keamanan khusus instalasi ini: token GitHub dan Vercel pernah dibagikan melalui kanal percakapan saat sesi penyiapan. Kedua token itu telah menjalankan tugasnya (push dan deploy selesai), sehingga langkah penutup yang benar sekarang adalah MEROTASI keduanya: GitHub - Settings - Developer settings - tokens - hapus token lama, buat baru bila masih dibutuhkan; Vercel - Account Settings - Tokens - hapus dan buat baru. Rotasi memutus akses bagi siapa pun yang kebetulan membaca kanal tersebut, dan tidak mengganggu deployment yang sudah hidup.

4. Prosedur Deploy Komponen

Bab ini adalah urutan kerja go-live dari nol sampai sistem hidup, disesuaikan dengan keadaan riil: aplikasi Next.js utama dan PWA standalone SUDAH berjalan di Vercel, jadi pekerjaan operator dimulai dari backend GAS, lalu menghubungkan klien ke backend tersebut. Urutan mengikuti ketergantungan nyata: GAS harus hidup lebih dulu karena dua klien dan firmware membutuhkan URL deployment-nya. Setiap langkah memuat klik atau perintah yang persis plus tanda berhasil. Total waktu realistis untuk instalasi pertama: 30-60 menit termasuk menunggu proses deploy Google dan propagasi DNS.

- **Prasyarat akun:** akun Google (gratis, untuk Apps Script - Bab 3.7 baris 1). Akun GitHub dan Vercel hanya perlu bila akan mengelola/memperbarui kode (Bab 6.2) atau mengubah env var lewat dashboard; penggunaan harian tidak membutuhkan keduanya.
- **Prasyarat perangkat:** satu papan ESP32 (DevKit v1 atau serupa), kabel data USB, sensor DHT22 + sensor tanah kapasitif (opsional - mode simulasi tidak membutuhkannya), WiFi 2,4 GHz.
- **Prasyarat perangkat lunak:** Node.js 18+ di mesin operator (untuk alat injeksi/verifikasi), Arduino IDE 2.x dengan board core arduino-esp32 2.0.4+ dan pustaka ArduinoJson, browser desktop/Android terbaru (Chrome/Edge/Firefox).
- **Prasyarat berkas:** clone dua repositori (git clone https://github.com/desvandi/plts_monitor_PWA_only dan https://github.com/desvandi/plts_monitor_firmware-code.gs-etc, letakkan bersebelahan) dan folder rahasia deploy/ berisi vapid-keys.json, device-token.txt, script-properties.txt. Bila folder rahasia belum ada, buat ulang isinya dengan dua perintah di Bab 4.3.

4.1 Komponen GAS (backend push) - satu-satunya langkah yang tersisa

Backend GAS adalah komponen pertama karena dua klien dan firmware mengarah ke URL deployment-nya. Seluruh proses dilakukan di peramban pada editor Apps Script, tanpa instalasi apa pun di mesin operator. Sumber berkas: push-alarm/gas/ pada repo [plts_monitor_firmware-code.gs-etc](https://github.com/desvandi/plts_monitor_firmware-code.gs-etc). Ikuti enam langkah berikut secara berurutan; tanda berhasil tiap langkah dicantumkan di akhir butirnya.

- **Langkah 1 - buat proyek:** buka script.google.com (login akun Google Anda), klik Proyek baru (New project). Beri nama, misalnya MonitorIoT-Push. Editor terbuka dengan satu berkas bernama Code.gs.
- **Langkah 2 - tempel dua berkas:** dari repo, salin seluruh isi push-alarm/gas/Code.gs ke berkas Code.gs bawaan; klik plus (+) di samping Files - pilih Script - beri nama WebPushCore - tempel seluruh isi push-alarm/gas/WebPushCore.gs. Simpan (Ctrl+S). Kedua berkas HARUS terpisah dalam satu proyek yang sama - WebPushCore berisi pustaka krypto yang dipanggil Code.gs.
- **Langkah 3 - isi Script Properties:** klik Project Settings (ikon roda gigi) - bagian Script Properties - Add script property sebanyak tiga kali: VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, FW_DEVICE_TOKEN, dengan nilai baris-per-baris dari deploy/script-properties.txt (kamus lengkap - termasuk properti opsional VAPID_SUBJECT - ada di Bab 3.2). Bila folder rahasia tidak ada, jalankan dulu dua perintah pada Bab 4.3 langkah 1.

- **Langkah 4 - deploy Web App:** Deploy (tombol biru kanan-atas) - New deployment - ikon roda gigi kecil - pilih tipe Web app. Description: bebas; Execute as: Me; Who has access: Anyone. Klik Deploy, lalu Authorize access - pilih akun Anda - lanjutkan melewati peringatan "app not verified" (Advanced - Go to project) - Izinkan. Layar izin hanya muncul sekali.
- **Langkah 5 - salin URL deployment:** dialog sukses menampilkan Web app URL berformat `https://script.google.com/macros/s/<ID>/exec` - klik Salin. URL inilah API_BASE untuk panel Settings aplikasi Next.js, --url untuk injektor PWA, dan GAS_URL firmware. Nilainya semi-publik (aman ditanam di klien) tapi panjang - simpan di catatan Anda.
- **Langkah 6 - uji nyala:** buka URL deployment + akhiran `?action=snapshot` di peramban - harus membalas JSON status perangkat dan alarm aktif (boleh kosong karena firmware belum hidup). Tanda gagal yang benar: halaman meminta login (berarti Who has access belum Anyone) atau galat script (berkas belum lengkap).

Bila organisasi memakai clasp (alat resmi Google untuk Apps Script dari terminal), kedua berkas .gs dapat didorong langsung dari folder gas/ repo: `clasp push` setelah clasp login dan mengisi .clasp.json (scriptId dari Project Settings). Pastikan .clasp.json tidak pernah ter-commit (sudah masuk .gitignore repo). Untuk instalasi pertama, jalur editor manual di atas sepenuhnya cukup dan tidak membutuhkan kredensial tambahan. Perubahan kode GAS berikutnya cukup dilakukan lewat Deploy - Manage deployments - ikon pensil - Version: New version - Deploy agar URL tetap sama.

4.2 Menghubungkan aplikasi Next.js utama ke GAS

Aplikasi sudah LIVE di `https://jmsepltsmonitoring.vercel.app` dengan kunci VAPID publik ter-bake dari env var Vercel; satu-satunya nilai yang belum ada adalah URL deployment GAS Anda (Bab 4.1 langkah 5). Ada dua jalur sah - pilih SALAH SATU (jalur A lebih cepat dan tidak memerlukan redeploy; jalur B membuat konfigurasi bawaan untuk semua pengguna baru).

- **Jalur A - panel Settings runtime (tanpa redeploy, per peramban):** buka `https://jmsepltsmonitoring.vercel.app` - masuk ke menu Settings - cari panel "Server Push Alarm (GAS PushService)" - isi kolom URL Web App GAS dengan URL `/exec` Anda - kunci publik VAPID biasanya sudah terisi bawaan build (bila kosong, tempel kunci publicKey dari `deploy/vapid-keys.json`) - Simpan. Uji: ketuk tombol uji push di panel yang sama - notifikasi uji harus muncul dalam beberapa detik.
- **Jalur B - env var Vercel untuk semua pengguna (perlu redeploy):** login vercel.com - proyek `jmse_plts_monitoring` - Settings - Environment Variables - tambah `NEXT_PUBLIC_PUSH_API_BASE` = URL `/exec` Anda (centang Production+Preview+Development) - Add - Deployments - deploy teratas - menu tiga titik - Redeploy. Tunggu READY, lalu buka aplikasi: panel Settings menampilkan "default build-time tersedia" dan pengguna baru tidak perlu mengisi apa pun.
- **Verifikasi koneksi (kedua jalur):** panel Settings menampilkan status langganan aktif setelah Anda menekan "Aktifkan notifikasi"; tombol uji push membalas sukses; snapshot aplikasi menampilkan data perangkat begitu firmware hidup (Bab 4.4).

4.3 PWA standalone: injeksi konfigurasi dan hosting

PWA standalone sudah LIVE di <https://plts-monitor-push-alarm.vercel.app>. Subbab ini diperlukan bila Anda ingin memakainya serius (mengisi konfigurasinya) atau menempatkannya di hosting lain. Berbeda dari aplikasi Next.js yang bisa dikonfigurasi runtime, PWA standalone menanam konfigurasi saat build - karena itu injektor adalah satu-satunya cara yang disarankan: ia memvalidasi format URL dan pasangan kunci VAPID SEBELUM menulis, menanam nilai ke config.js, sw.js, dan firmware secara atomik, lalu memverifikasi hasil tulis.

```
# Langkah 1 - siapkan folder rahasia (lewati bila deploy/ sudah ada):
cd plts_monitor_firmware-code.gs-etc/push-alarm
node tools/generate-vapid-keys.js --save
node tools/prep-deploy-secrets.js --secrets-dir ../../download/deploy

# Langkah 2 - tanam konfigurasi ke salinan build (templat tetap murni):
node tools/apply-deploy-config.js \
  --pwa-dir ../plts_monitor_PWA_only/pwa-push-alarm \
  --fw-dir  firmware/MonitorIoT_Firmware \
  --url     "https://script.google.com/macros/s/ID_DEPLOYMENT_ANDA/exec" \
  --keys    ../../download/deploy/vapid-keys.json \
  --token   ../../download/deploy/device-token.txt \
  --ssid    "NAMA_WIFI_ANDA" --pass "PASSWORD_WIFI_ANDA" \
  --out     build

# Langkah 3 - gerbang wajib, keluaran harus SIAP DEPLOY:
node tools/verify-deployment.js \
  --config build/pwa-push-alarm/js/config.js \
  --sw     build/pwa-push-alarm/sw.js \
  --keys   ../../download/deploy/vapid-keys.json
```

Perintah di atas menghasilkan build/pwa-push-alarm (PWA siap hosting) dan build/MonitorIoT_Firmware (sketch siap dibuka Arduino IDE dengan URL, token, dan WiFi tertanam). Argumen --ssid/--pass opsional: bila dilewatkan, isi kredensial WiFi langsung di IDE. Tanpa --out, injektor menulis DI TEMPAT pada templat - jangan dipakai pada clone repositori agar riwayat git tetap bersih dari nilai produksi. Gerbang verify-deployment memeriksa API_BASE config.js identik dengan sw.js, format URL GAS sah, kunci publik berformat P-256 65 byte, dan membuktikan secara matematis bahwa kunci publik adalah turunan kunci privat (kontrak K7). Bila keluarannya bukan SIAP DEPLOY, perbaiki sesuai pesan galatnya; jangan lanjut ke hosting.

Hosting build PWA (berkas statis): unggah ISI folder build/pwa-push-alarm ke hosting HTTPS mana pun. Contoh jalur: Netlify Drop (app.netlify.com/drop - seret folder, URL aktif dalam detik); Cloudflare Pages/Firebase Hosting (hubungkan repo atau unggah folder); atau Vercel CLI: `vercel deploy --prod` dari dalam folder build. Persyaratan teknis hanya dua: HTTPS aktif (push menolak HTTP) dan sw.js disajikan dari root scope dengan header cache no-cache (vercel.json pada repo sudah mengaturnya untuk Vercel). Verifikasi pasca-hosting: dasbor sensor merender, lalu DevTools - Application - Service Workers menampilkan sw.js activated.

4.4 Firmware ESP32

- **Langkah 1 - buka sketch:** Arduino IDE - File - Open - pilih build/MonitorIoT_Firmware/MonitorIoT_Firmware.ino hasil injeksi (atau sketch langsung dari repo bila

mengisi WiFi manual).

- **Langkah 2 - cek papan dan pustaka:** Boards Manager pasang arduino-esp32 2.0.4+; Library Manager pasang ArduinoJson (dan Adafruit DHT + Unified Sensor bila memakai sensor nyata).
- **Langkah 3 - pilih mode sensor:** biarkan `SENSOR_MODE_SIMULATED` aktif untuk uji pertama tanpa perangkat sensor; aktifkan `SENSOR_MODE_DHT22` saat perangkat sudah terpasang.
- **Langkah 4 - upload:** pilih board ESP32 Dev Module dan port USB, klik Upload. Buka Serial Monitor 115200 baud - baris pertama harus membaca MonitorIoT Firmware v1.1.0 (ESP32).
- **Langkah 5 - konfirmasi jalur:** dalam 1-2 menit muncul baris [KIRIM] ok - laporan diterima GAS. Buka aplikasi Next.js atau PWA: nilai sensor muncul di dasbor dalam interval polling. Tanda gagal umum: 401/403 (token tidak cocok - cek `FW_DEVICE_TOKEN` Script Properties vs `DEVICE_TOKEN` sketch) atau -1 connection (WiFi/TLS - lihat Tabel 8.1).

4.5 Aktivasi langganan push pertama

Langkah terakhir go-live adalah menghubungkan pelanggan ke layanan, per peramban yang ingin menerima alarm. Di aplikasi Next.js utama: buka Settings - panel Server Push Alarm - aktifkan notifikasi (izin peramban diminta dari gestur ketukan - bukan otomatis), lalu ketuk tombol uji push. Di PWA standalone: ketuk tombol Aktifkan Notifikasi Alarm pada dasbor. Uji ketahanan jalur tertutup: dari editor GAS pilih fungsi `simulateAlarmPush` pada dropdown fungsi lalu Run - satu notifikasi berjudul Uji Push Berhasil harus muncul WALAU tab aplikasi sudah ditutup. Bila semua jalur di atas hijau, sistem resmi hidup; jalankan uji acceptance Bab 5 untuk menutup rilis.

5. Verifikasi dan Uji Acceptance

Verifikasi go-live dibagi dua lapis: gerbang otomatis (alat menilai mesin - tidak ada ruang salah paham) dan uji acceptance manual (operator menilai pengalaman pengguna nyata pada perangkat nyata). Keduanya wajib hijau sebelum sistem dinyatakan beroperasi. Gerbang otomatis juga menjadi protokol regresi: jalankan ulang setiap kali komponen diubah, sebelum push, dan sebelum flash ulang firmware. Tabel 5.1 memuat dua kelompok gerbang: sisi toolkit (dari clone folder push-alarm) dan sisi aplikasi Next.js (dari clone repo aplikasi).

5.1 Gerbang otomatis pra-go-live

Gerbang	Perintah	Kriteria lulus
Konsistensi & kunci (K7)	node tools/verify-deployment.js --config ... --sw ... --keys ... (folder push-alarm)	Keluaran SIAP DEPLOY, exit 0
Regresi tiga komponen	bash tests/run-all.sh (dari folder push-alarm, kedua repo bersebelahan)	3/3 suite lulus: 35+17+151 = 203 asersi
Rahasia & higienitas repo	node tools/prepush-audit.js --repos <folder-2-repo>	PRE-PUSH: LULUS, exit 0
Rantai TLS firmware	node tools/verify-tls-ca-embed.js (folder push-alarm)	Kedua root byte-identik + valid openssl
Tipe & lint aplikasi Next.js	npm run typecheck; npm run lint (repo aplikasi)	0 galat tipe; 0 error lint (warning warisan diizinkan)
Unit aplikasi Next.js	npm test (vittest)	106/106 lulus (termasuk 25+ uji darurat + aliran energi baru)
Lapisan darurat GAS	node scripts/test_emergency_gas.js (repo firmware)	46/46 lulus
Logika darurat firmware	python3 scripts/test_emergency_firmware_logic.py (repo firmware)	34/34 lulus
Kontrak GAS inti	node scripts/test_gas_contract.js (repo firmware)	75/75 lulus
Kontrak firmware wave-6	node scripts/test_wave6_firmware_completion.js (repo firmware)	45/45 lulus (firmware produksi v1.6.3)
Logika firmware Python	python3 -m pytest scripts/ (repo firmware)	83 lulus
Build aplikasi Next.js	npm run build	Build sukses, public/sw.js memuat handler push + config ter-bake bila env var diset
Deployment hidup	curl -s https://jmsepltsmonitoring.vercel.app/sw.js (periksa isinya)	HTTP 200; isi memuat handler push; kunci VAPID publik ter-bake; header cache sw.js no-cache

Tabel 5.1 - Gerbang otomatis dan kriteria lulusnya.

Suite regresi layak diulang beberapa kali berturut-turut: temuan X-8 pada audit silang (skalar privat pendek dari ekspor DER, kejadian probabilistik sekitar 0,4 persen pasangan kunci) hanya tertangkap oleh loop beruntun. Demikian pula bug decoder base64url yang ditemukan pada integrasi Next.js hanya muncul pada kunci acak berkarakter -/_ (sekitar 93 persen kunci acak gagal) - kini dikunci dua asersi regresi permanen. Protokol rilis menjalankan krypto dan audit silang minimal empat putaran penuh tanpa satu pun kegagalan sebelum status produksi diberikan.

5.2 Skenario acceptance manual (S1-S5)

ID	Skenario	Langkah	Hasil yang diharapkan
S1	Langganan & izin	Buka aplikasi Next.js - Settings - panel push - aktifkan notifikasi, tutup tab (ulangi di PWA standalone bila dipakai)	Status push aktif; izin Notification = granted; langganan tercatat di sisi GAS (registry bertambah)
S2	Push saat aplikasi tertutup	Dari editor GAS jalankan simulateAlarmPush dengan semua tab tertutup	Notifikasi tampil dengan judul, isi, badge; klik membuka aplikasi langsung ke halaman Alarms (deep-link ?view=alarms&from;=push)
S3	ACK & deep-link	Ketuk tombol Tandai Ditangani pada notifikasi	Aplikasi terbuka menampilkan alarm ditandai; status di snapshot GAS berubah menjadi acknowledged (idempoten)
S4	Tepi alarm tepat satu kali	Naikkan SIM_TEMP_CENTER melewati 40 dua siklus, lalu kembalikan	Push alarm SEKALI saat tepi naik; push PULIH SEKALI saat pulih; tidak ada push berulang selama level tetap alarm
S5	Ketahanan jaringan	Matikan WiFi/router 5 menit lalu nyalakan kembali	Firmware backoff eksponensial maks 8 menit, reconnect otomatis, laporan lanjut tanpa reboot; alarm yang terjadi saat offline tetap terkirim saat koneksi pulih

Tabel 5.2 - Skenario acceptance go-live.

Skenario S2 adalah alasan keberadaan seluruh arsitektur ini - jalankan minimal pada satu perangkat Android dan satu browser desktop karena implementasi push service berbeda antar platform. Untuk iOS, pastikan PWA dipasang ke Layar Utama lebih dulu (Add to Home Screen) karena Safari iOS menolak push dari tab biasa. Skenario S4 menguji jantung kontrak K6: firmware hanya melaporkan level, GAS yang memutuskan tepi - bila push ganda atau hilang, hentikan rilis dan periksa firmware tidak memanggil endpoint selain ingest.

5.3 Skenario acceptance darurat WAVE-7 (S6-S10)

Lima skenario berikut hanya relevan bila Anda memasang aktuatur darurat (firmware-generic 1.6.0 + relai + E-stop). Jalankan semuanya saat KOMISIONING, sebelum sistem dinyatakan beroperasi - aktuatur keselamatan yang tidak diuji adalah bahaya terselubung. Amankan beban sensitif-glitch dengan UPS sebelum mulai: skenario S6 dan S8 memutus daya sesaat sesuai desain. Panel uji ada di menu Kontrol Darurat (ikon perisai) aplikasi utama.

ID	Skenario	Langkah	Hasil yang diharapkan
S6	EMERGENCY STOP jarak jauh	Isi ADMIN_TOKEN (halaman /setup); buka menu Kontrol Darurat; tekan EMERGENCY STOP lalu ketik kata STOP sebagai konfirmasi	Sistem TERISOLASI dalam 15 detik; kolom emg_state telemetry berubah; kejadian DISARMED muncul di EmergencyEvents; alert Telegram terkirim (bila dikonfigurasi)
S7	ARM + gate keselamatan	Sebelum pemicu dikembalikan normal, tekan ARM; perbaiki pemicu, tekan ARM lagi setelah masa pulih	ARM pertama DITOLAK dengan alasan (picu aktif / masa pulih) tampil di toast; ARM kedua diterima setelah kondisi aman - sistem RUN kembali; baris queue APPLIED
S8	Trip otomatis ambang	Editor ambang: naikan vbatLowV di atas tegangan aktual (mis. 54 V), tunggu debounce 3 siklus, kembalikan	Trip dengan alasan VBAT_LOW di panel; kejadian TRIP di EmergencyEvents; setelah histeresis terlewati + masa pulih, ARM diterima
S9	E-stop fisik + indikasi	Tekan tombol E-stop NC, amati panel Kontrol Darurat, putar untuk melepas, tekan ARM	Sistem TERISOLASI seketika (jalur hardware); indikator E-stop menyala di panel (pin sense); lepas tombol TIDAK menyalakan ulang - hanya ARM yang bisa
S10	Perintah basi (TTL)	Matikan WiFi perangkat, kirim EMERGENCY STOP, tunggu 11 menit, nyalakan WiFi	Perintah EXPIRED di queue (tidak diterapkan diam-diam saat perangkat online kembali); sistem tetap RUN; kejadian tercatat di log

Tabel 5.3 - Skenario acceptance darurat WAVE-7 (S6-S10).

Skenario S7 dan S9 MENGUJI FILOSOFI, bukan hanya fungsi: sistem yang menolak menyala saat belum aman dan sistem yang tetap mati setelah E-stop dilepas adalah BUKTI arah gagal-aman bekerja. Bila salah satu skenario gagal, jangan hanya ARM dan lanjut - hentikan komisioning, periksa wiring relai dan posisi E-stop, dan ulangi dari S6. Kuota GAS tidak terpengaruh acceptance ini: perintah menumpang piggyback TELEMETRI dan poll 15 detik yang hanya aktif saat backoff (Bab 2.4).

6. Push Repositori dan Manajemen Rahasia

6.1 Klasifikasi rahasia vs nilai publik

Nilai	Klasifikasi	Tempat hidup yang sah
VAPID_PRIVATE_KEY	RAHASIA	Hanya Script Properties GAS; berkas vapid-keys.json di folder rahasia lokal (izin 600).
VAPID_PUBLIC_KEY	PUBLIK	Script Properties GAS + env var Vercel + panel Settings + js/config.js PWA - aman ditanam di klien mana pun.
FW_DEVICE_TOKEN	RAHASIA	Script Properties GAS + konstanta DEVICE_TOKEN firmware yang sudah ter-flash (tidak di-commit).
URL deployment GAS	SEMI-PUBLIK	Ditanam di klien (siapa pun yang membuka aplikasi bisa melihatnya); tidak perlu dirahasiakan ketat, tetapi jangan disebar tanpa perlu.
Kredensial WiFi	RAHASIA LOKAL	Hanya di sketch yang di-flash / memori flash ESP32.
Root CA Google	PUBLIK	Tertanam di firmware; dapat diverifikasi siapa pun.
Token GitHub (PAT)	RAHASIA AKUN	Manajer kredensial / sesaat di URL push lalu dihapus; tidak pernah dalam berkas ter-commit atau repos.conf.
Token Vercel	RAHASIA AKUN	Variabel lingkungan mesin operator (VERCEL_TOKEN) / kredensial CLI; tidak pernah dalam berkas ter-commit.
NEXT_PUBLIC_PUSH_* (env var Vercel)	PUBLIK by-design	Dashboard Vercel proyek; nilai memang untuk dikonsumsi klien. Akses DASHBOARD-nya yang harus dibatasi.

Tabel 6.1 - Klasifikasi seluruh nilai sensitif sistem.

Prinsip tunggal yang menjaga seluruh klasifikasi di atas: repositori SELALU berisi templat placeholder atau konfigurasi runtime - nilai produksi hanya hidup di Script Properties GAS, env var Vercel, memori firmware, isian panel Settings peramban, atau build hasil injektor yang tidak pernah di-commit (folder build/, staging/, deploy/ semuanya masuk .gitignore). Gerbang prepush-audit.js memindai pelanggaran prinsip ini pada setiap berkas terlacak - nilai eksaklit, pola token 43-karakter base64url, blok kunci privat PEM, dan URL deployment non-placeholder - dan memblokir push bila ada satu saja temuan.

6.2 Prosedur push pembaruan ke GitHub

Kedua repositori sudah terdorong dan hidup; prosedur ini dipakai untuk SETIAP pembaruan kode berikutnya. Alur kerja standar: ubah kode - jalankan gerbang mutu Tabel 5.1 - commit - push. Bila memakai HTTPS, token akses personal dibutuhkan sesaat (pembuatan token: Bab 3.7). Pola amannya: sisipkan token di URL hanya saat push, lalu kembalikan remote ke bentuk tanpa token.

```

cd plts_monitor_PWA_only          # atau repo firmware-code.gs-etc
# 1) Gerbang mutu sebelum commit (lihat Tabel 5.1)
# 2) Commit
git add -A && git commit -m "fitur/perbaikan: ringkas"

# 3) Push dengan token disisipkan sesaat (ganti <PAT>):
git remote set-url origin https://<PAT>@github.com/desvandi/plts_monitor_PWA_only.git
git push origin main

# 4) Kembalikan remote tanpa token SEGERA:
git remote set-url origin https://github.com/desvandi/plts_monitor_PWA_only.git

# 5) Audit higienitas kapan pun:
node tools/prepush-audit.js --repos <folder-2-repo>

```

- Push ke `plts_monitor_PWA_only` otomatis memicu deploy ulang proyek Vercel `jmse_plts_monitoring` (auto-deploy dari branch `main`) - pantau statusnya di tab `Deployments`; URL tidak berubah. Bila env var `NEXT_PUBLIC_*` ikut diubah, cukup redeploy biasa: nilai baru ter-bake pada build berikutnya.
- SSH (`git@github.com:...`) menghindari PAT sama sekali untuk mesin yang sering push: pasang kunci SSH sekali lewat `Settings` - `SSH and GPG keys`.
- `git pull --rebase origin main` bila repo jauh lebih maju (misal diperbarui dari mesin lain).
- Pembaruan backend GAS TIDAK lewat git: edit di editor Apps Script lalu `Deploy` - `Manage deployments` - `New version` (URL tetap sama).

6.3 Prosedur rotasi saat kebocoran

Bila salah satu rahasia terdeteksi bocor (ter-commit, terkirim ke log, tersebar, atau sekadar dibagikan lewat kanal percakapan), jangan panik - seluruh nilai dirancang dapat dirotasi tanpa membangun ulang sistem. Urutan rotasi kunci VAPID:

1. `generate-vapid-keys.js --save` (pasangan baru), simpan ke folder rahasia dengan izin 600.
2. Perbarui Script Properties GAS: `VAPID_PUBLIC_KEY` dan `VAPID_PRIVATE_KEY`.
3. Perbarui env var Vercel `NEXT_PUBLIC_PUSH_VAPID_PUBLIC_KEY` lalu `Redeploy`; (opsional) minta pengguna lama membuka panel `Settings` agar konfigurasi runtime ikut diperbarui; pelanggan lama otomatis berlangganan ulang melalui event `pushsubscriptionchange` saat push berikutnya ditolak.
4. Jalankan `verify-deployment.js` untuk membuktikan pasangan baru konsisten, lalu uji push untuk memastikan pelanggan menerima.

Rotasi nilai lain: token perangkat (buat token baru dengan `prep-deploy-secrets.js`, perbarui `FW_DEVICE_TOKEN` di Script Properties, injeksi ulang firmware, flash ulang ESP32; bila memakai `FW_DEVICE_TOKENS` cukup ganti baris perangkat yang bocor). Token GitHub: `Settings` - `Developer settings` - hapus token lama, buat baru (token lama langsung mati saat dihapus). Token Vercel: `Account Settings` - `Tokens` - hapus, buat baru. Untuk riwayat git yang ternyata memuat rahasia di repo PUBLIK: rotasi nilai terlebih dahulu (kunci lama sudah tidak bernilai setelah dirotasi), baru bersihkan riwayat dengan `git filter-repo` atau `BFG` bila diinginkan.

6.4 Cadangan lokal dan disiplin harian

Simpan folder rahasia deploy/ di dua tempat: folder kerja (izin 600) dan satu cadangan terenkripsi (vault berbasis kata sandi, atau arsip terenkripsi di penyimpanan awan pribadi). Kehilangan kunci privat VAPID tidak membocorkan apa pun, tetapi memaksa rotasi dan sesi berlangganan ulang semua pelanggan; kehilangan token perangkat mengunci firmware dari GAS. Sebelum setiap push: pastikan git status bersih dan biarkan prepush-audit bekerja - kebiasaan lima detik yang mencegah insiden berbulan-bulan.

7. Operasional Pasca Go-Live

7.1 Pemantauan rutin

Objek	Jalur	Irama
Firmware sehat	Serial Monitor: baris [KIRIM] ok berkala; LED indikator lokal	Saat kunjungan / uji meja
GAS sehat	Editor GAS - Executions (log sukses/gagal per permintaan)	Mingguan
Endpoint hidup	Peramban: URL deployment + ?action=snapshot	Mingguan atau pasca insiden
Aplikasi hidup	Buka https://jmsepltsmonitoring.vercel.app ; dashboard Vercel - tab Deployments (status READY)	Mingguan atau pasca pembaruan
Push hidup	Tombol uji push pada panel Settings / PWA - dibatasi 60 detik	Bulanan
Registri langganan	Snapshot GAS mencerminkan langganan aktif; prune otomatis 90 hari	Bulanan

Tabel 7.1 - Jadwal pemantauan minimum.

7.2 Kuota GAS dan penyetelan kapasitas

Google Apps Script menerapkan kuota harian UrlFetchApp (panggilan keluar) dan batas eksekusi 6 menit per run. Dengan pengaturan bawaan, satu perangkat mengirim 1.440 laporan/hari dan setiap laporan memicu pengiriman push hanya saat tepi alarm (bukan tiap laporan), sehingga konsumsi kuota tetap rendah. Bila armada perangkat tumbuh: turunkan frekuensi laporan (REPORT_INTERVAL_MS), naikkan BATCH_SIZE hanya bila jumlah pelanggan melewati ratusan, dan biarkan BATCH_SIZE tetap 100 untuk instalasi kecil. Endpoint testPush dibatasi satu permintaan per 60 detik untuk mencegah banjir push dari pengguna internet. Perbandingan lengkap kuota terhadap beban instalasi dua HP + satu modul ada di Lampiran A.

7.3 Pemeliharaan berkala dan pembaruan

- **Rotasi kunci VAPID:** direkomendasikan tahunan (opsional; segera setelah indikasi bocor). Prosedur lengkap Bab 6.3.
- **Rotasi token GitHub/Vercel:** rotasi kebiasaan baik tahunan; WAJIB segera untuk token yang pernah dibagikan lewat kanal percakapan (lihat catatan penutup Bab 3.7).
- **Root CA Google:** root tertanam berlaku hingga Juni 2036; Google dapat merotasi lebih awal. Regenerasi: perbarui tools/roots.pem dari pki.goog, jalankan gen-tls-ca-embed.js, verifikasi dengan verify-tls-ca-embed.js, tempel blok baru ke sketch, naikan FW_VERSION, flash ulang.
- **Pembaruan firmware:** selalu naikan FW_VERSION (terlapor ke GAS sebagai device.fw) agar jejak armada dapat diaudit dari log laporan.
- **Pembaruan aplikasi Next.js:** commit dan push ke main - auto-deploy Vercel bekerja; pantau Deployments hingga READY lalu uji cepat S1-S2. Bila memakai env var baru: setel dulu di Settings lalu Redeploy.
- **Pembaruan PWA standalone:** injeksi ulang build, hosting ulang, naikan APP_VERSION di config.js untuk cache busting.
- **Menambah perangkat:** baca FW_DEVICE_TOKENS (Bab 3.2) - satu pasangan deviceId+token per perangkat; DEVICE_ID firmware harus cocok persis; tidak perlu mengubah kode GAS.

7.4 Operasional darurat WAVE-7 (ARM/DISARM dan ambang)

Menu Kontrol Darurat di aplikasi utama adalah pusat kendali darurat sehari-hari: status relai (RUN / TERISOLASI / TIDAK DIKETAHUI - firmware pra-1.6.0 jujur dilaporkan TIDAK DIKETAHUI), alasan trip terakhir, total trip, indikator jalur E-stop fisik, tombol ARM (satu klik, perangkat boleh menolak dengan alasan), tombol EMERGENCY STOP (konfirmasi mengetik kata STOP), editor 12 ambang, dan riwayat kejadian. ADMIN_TOKEN per perangkat diisikan di halaman /setup dan disimpan HANYA di localStorage peramban operator - tidak pernah di repositori atau build; mengosongkannya langsung mematikan tombol-tombol darurat (fail-closed).

Saat trip terjadi, notifikasi menyala berlapis: status panel berubah TERISOLASI dengan alasan, kejadian TRIP/ESTOP tercatat di sheet EmergencyEvents, dan pesan Telegram terkirim bila TELEGRAM_BOT_TOKEN/CHAT_ID terisi (cooldown 2 menit per topik). Rilis E-stop fisik TIDAK menyalakan ulang sistem - hanya ARM dari operator, dan firmware menolaknya bila kondisi belum aman. Diagram Aliran Energi ikut jujur: sistem TERISOLASI membekukan seluruh aliran.

Disiplin ambang: ubah dari editor panel (bukan sunting sheet manual) agar validasi 3 lapis berjalan (PWA clamp - GAS range-check - firmware range-check); perangkat menyimpan hasilnya di LittleFS dan field tak dikenal dibuang. Pin relai/E-stop bisa dipindah lewat CONFIG tanpa reflash - berguna saat revisi wiring; setelah memindah pin, ulangi skenario S9 untuk memastikan sense E-stop membaca jalur yang benar.

8. Penanganan Masalah

Tabel berikut memetakan gejala yang paling sering dilaporkan ke penyebab dominan dan penanganan terverifikasi. Baris pertama selalu periksalah: lebih dari separuh laporan masalah push terselesaikan di baris tabel nomor satu dan dua. Baris khusus aplikasi Next.js ditandai pada kolom gejala.

Gejala	Penyebab dominan	Penanganan
Notifikasi tidak muncul saat aplikasi tertutup	Service worker tidak aktif / PWA dibuka di iOS tanpa Add to Home Screen / izin notifikasi diblokir	DevTools - Application - Service Workers pastikan sw.js activated; iOS: pasang ke Layar Utama (minimal iOS 16.4); Setelan situs - izinkan notifikasi; uji ulang S2
(Next.js) Panel Settings: "belum dikonfigurasi" walau env var sudah diisi	Env var ditambah SETELAH build terakhir - nilai NEXT_PUBLIC_* di-bake saat build	Vercel - Deployments - deploy teratas - Redeploy; verifikasi: sw.js memuat kunci (Bab 5.1 baris terakhir)
(Next.js) Config panel tersimpan tapi tidak dipakai SW	Sinkronisasi terjadi saat muat halaman; SW lama masih aktif	Muat ulang halaman sekali; cek DevTools - Application - IndexedDB - database push-alarm berisi config; hard refresh bila perlu
Push masuk tetapi isi kosong	Payload gagal didekripsi peramban (framing) atau melebihi 4096 byte	GAS otomatis fallback ke latestAlarm; pastikan WebPushCore.gs tidak dimodifikasi; cek MAX_PAYLOAD_CHARS=3000
Subscribe gagal: kunci tidak valid	Kunci VAPID publik tidak didekode benar (karakter - / _) atau terpotong saat disalin	Salin ulang 87 karakter utuh dari vapid-keys.json tanpa spasi/ tanda kutip; pastikan identik di GAS/Vercel/panel (K7)
verify-deployment: API_BASE masih placeholder	Injektor belum dijalankan / target --config salah	Jalankan ulang apply-deploy-config.js (Bab 4.3) lalu verifikasi ulang; pastikan path config/sw menunjuk build yang benar
verify-deployment: kunci tidak cocok / format salah	Public dan private key berasal dari pasangan berbeda, atau kunci privat bukan 32 byte	Pakai satu vapid-keys.json utuh (jangan salin per baris); regenerasi bila ragu lalu perbarui Script Properties
Firmware: kirim gagal berulang (-1 / connection)	WiFi salah, board core ESP32 lama (setCACert ganda), atau jaringan memutus TLS (proxy captive)	Cek SSID/password 2,4 GHz; naikan arduino-esp32 ke 2.0.4+; jaringan proxy: aktifkan TLS_SKIP_CERT_VERIFY (sadar risiko MITM) hanya selama uji
Firmware: GAS menolak laporan (401/403)	Token atau deviceId tidak cocok dengan Script Properties	Samakan DEVICE_TOKEN dengan FW_DEVICE_TOKEN; bila memakai FW_DEVICE_TOKENS pastikan DEVICE_ID firmware terdaftar persis
GAS log: kuota / timeout	Kuota UrlFetchApp habis atau run melebihi 6 menit	Turunkan REPORT_INTERVAL_MS; pastikan BATCH_SIZE 100; periksa jumlah pelanggan (prune 90 hari bekerja); lihat Lampiran A untuk perbandingan kuota
Alarm push terkirim dua kali	Firmware atau pihak lain memanggil endpoint selain ingest, ATAU perangkat yang sama subscribe di dua aplikasi sekaligus (Bab 2.3), atau laporan tidak menyertakan flag alarm	Pastikan firmware 1.1.0 menyertakan flag alarm per sensor; jaga prinsip pengirim tunggal; satu perangkat cukup satu langganan push; GAS fallback THRESHOLDS hanya untuk firmware pra-flag

Gejala	Penyebab dominan	Penanganan
testPush selalu ditolak	Rate-limit 60 detik sedang aktif (perilaku benar)	Tunggu interval; atau set <code>TEST_PUSH_MIN_INTERVAL_MS</code> lebih kecil HANYA di meja uji jaringan tertutup
Semua langganan mati setelah ganti kunci	Rotasi kunci membuat langganan lama tidak cocok	Perilaku normal: klien otomatis resubscribe via <code>pushsubscriptionchange</code> ; minta pengguna membuka aplikasi sekali bila tidak aktif dalam 24 jam
Aplikasi Next.js tidak bisa subscribe sama sekali	Konfigurasi belum diisi (URL GAS kosong) atau izin peramban ditolak permanen	Isi URL GAS lewat panel Settings / env var (Bab 4.2); reset izin notifikasi di ikon kunci address bar lalu aktifkan ulang dari tombol (gestur)
Dasbor menampilkan nilai 0 aneh	Firmware offline / sensor gagal baca (NAN) / kuota habis	Cek banner status koneksi; nilai null di-render sebagai tanda baca gagal (kontrak X-3), bukan 0 palsu

Tabel 8.1 - Pemetaan gejala, penyebab, dan penanganan.

9. Sertifikasi Kesiapan Produksi

9.1 Bukti audit berlapis

Lapis	Bukti	Hasil
Kripto inti (test-webpush-core.js)	Vektor RFC 4231/5869, NIST GCM, RFC 6979 P-256; uji acak vs Node/WebCrypto; skalar pendek X-8	35/35 lulus, stabil lintas putaran
PWA di peramban nyata (smoke-test-pwa.js)	Chromium headless: manifest, SW aktif, render, izin, handler push, konsol bersih	17/17 lulus
Kontrak lintas komponen (cross-audit-test.js)	K1-K8 Tabel 11: kode asli tiga komponen + mock push service yang mendekripsi aes128gcm seperti peramban nyata; kerasifikasi atob harness	151/151 lulus (19+9+29+8+9+47+17+13)
Aplikasi Next.js (vitest + tsc + eslint)	79 unit termasuk 17 asersi push-alarm (decoder, ACK, opsi notifikasi); 0 galat tipe; 0 error lint	79/79 + 0 + 0 lulus
Build produksi Next.js	next build sukses; sw.js memuat semua handler push; kunci VAPID ter-bake dari env var	LULUS - terverifikasi pada deployment live
Rantai TLS firmware (verify-tls-ca-embed.js)	GTS Root R1+R4 byte-identik pki.goog, valid openssl, dipinkan di suite K8	4/4 lulus
Pipa deployment (staging + injektor)	Injeksi 5 nilai, 5 verifikasi pasca-tulis, gerbang SIAP DEPLOY; dua uji negatif menolak URL tidak sah	LULUS
Higienitas repositori (prepush-audit.js)	Dua repositori GitHub: bebas rahasia eksaklit + pola; sinkron templat; git bersih; uji negatif menangkap seluruh vektor tanam rahasia	12/12 lulus, 0 gagal
Deployment publik	Push GitHub 2 repo; 2 proyek Vercel READY; verifikasi endpoint live: halaman, sw.js, manifest, ikon - semua 200	LIVE - terverifikasi

Tabel 9.1 - Bukti audit berlapis status rilis.

9.2 Batas peran alat vs operator

Status PRODUCTION GRADE pada bagian ini menyatakan: seluruh pekerjaan yang dapat dilakukan tanpa kredensial operator telah selesai dan diverifikasi - repositori terdorong, deployment hidup, konfigurasi bawaan ter-bake. Yang tetap milik operator adalah tindakan yang menuntut akun dan perangkat pribadi - bukan pekerjaan tertunda, melainkan batas peran yang wajar:

- Menempel dua berkas GAS + tiga Script Properties + menekan Deploy di akun Google milik operator (Bab 4.1).
- Menempel URL deployment GAS ke panel Settings atau env var Vercel (Bab 4.2, jalur A atau B).
- Mengisi kredensial WiFi lokasi dan menekan Upload di Arduino IDE (Bab 4.4).
- Menjalankan lima skenario acceptance pada perangkat nyata milik operator (Bab 5.2).

- Merotasi token GitHub/Vercel yang pernah dibagikan lewat kanal percakapan (Bab 3.7 catatan keamanan).

9.3 Verdict

PRODUCTION GRADE - deployment live, gerbang terbuka	203+79 asersi regresi + unit hijau	2+2 repo GitHub + proyek Vercel LIVE	Rp0 biaya seluruh platform (Lampiran A)
--	--	---	--

Sistem MonitorIoT (aplikasi Next.js utama + PWA standalone v2.0.0, backend GAS, firmware ESP32 v1.1.0) dinyatakan LULUS audit berlapis dan SIAP PRODUCTION GRADE pada 1 September 2026: delapan kontrak Tabel 11 terverifikasi mekanis (151 asersi silang termasuk lapis hardening K8), krypto inti teruji terhadap vektor standar industri, rantai TLS firmware terverifikasi byte-level terhadap sumber resmi Google, dua repositori GitHub bebas rahasia dengan gerbang audit otomatis, dua proyek Vercel melayani HTTPS dengan konfigurasi bawaan ter-bake, seluruh platform berjalan di paket gratis tanpa kartu kredit (Lampiran A), dan seluruh prosedur operasional dari nol hingga pemeliharaan terdokumentasi pada dokumen ini. Operator dapat melanjutkan langsung dari Bab 4.1.

Pernyataan operator	Tanda (paraf/tanggal)
Saya telah membaca dan memahami Bab 3-5.	
Backend GAS milik saya ter-deploy dan ?action=snapshot membalas JSON (Bab 4.1).	
URL GAS tertempel di panel Settings / env var Vercel (Bab 4.2).	
Gerbang otomatis 5.1 dan skenario acceptance 5.2 lulus pada instalasi saya.	
Rahasia deployment tersimpan aman; token yang pernah dibagikan sudah dirotasi (Bab 6.1/6.4).	
Saya memahami seluruh platform dipakai gratis Rp0 tanpa kartu kredit, beserta batas kuotanya (Lampiran A).	

Tabel 9.2 - Lembar tanda tangan go-live (operator).

10. Lampiran A - Biaya Rp0: Platform Gratis dan Kuota

Lampiran ini membuktikan dan mendokumentasikan janji biaya sistem: seluruh rantai berjalan di paket gratis platform publik, TANPA langganan, TANPA input kartu kredit di satu pun layanan. Bukti terkuatnya adalah fakta deployment: kedua proyek Vercel dan kedua repositori GitHub pada dokumen ini dibuat dan dioperasikan hanya dengan akun gratis dan token akses - tidak ada metode pembayaran yang pernah dimasukkan. Kebutuhan instalasi maksimum - dua HP penerima alarm dan satu modul monitoring - berada jauh di bawah batas paket gratis setiap lapis, dengan margin puluhan kali lipat.

10.1 Peta platform yang dipakai dan kuotanya

Tabel berikut memetakan enam lapis sistem ke platform yang dipakainya. Kolom "batas gratis" memuat angka perkiraan saat dokumen disusun - angka resmi terkini selalu bisa dicek gratis di halaman kuota masing-masing platform (dashboard Vercel memilikinya pada tab Usage/Limits; kuota GAS pada halaman resmi Apps Script Quotas). Kolom terakhir menghitung beban nyata instalasi dua HP + satu modul sehingga margin terlihat langsung.

Lapis	Platform yang dipakai	Batas paket gratis (perkiraan)	Beban kita (2 HP + 1 modul)
Hosting aplikasi & PWA	Vercel paket Hobby - dua proyek dalam satu akun; gratis selamanya untuk penggunaan personal; TANPA kartu kredit saat daftar	sekitar 100 GB transfer/bulan + batas build longgar; proyek personal tak dibatasi jumlahnya	puluhan MB/hari dari 2 HP - di bawah 1 persen kuota
Backend push + penyimpanan	Google Apps Script (akun Google gratis) - Web App, Script Properties, sheet opsional	sekitar 20.000 panggilan keluar (UrlFetch)/hari; 6 menit per eksekusi; kuota reset TIAP HARI	1.440 laporan masuk/hari dari modul + sekitar 40 panggilan push/hari - margin besar
Infrastruktur notifikasi	Push service peramban: FCM untuk Chrome/Android/Edge, APNs untuk Safari iOS 16.4+, Mozilla autopush untuk Firefox - dipilih OTOMATIS oleh browser	dikelola vendor peramban; bebas biaya dan bebas akun (kunci VAPID dibuat sendiri - TIDAK perlu proyek Firebase); pesan diteruskan selama TTL 24 jam	beberapa pesan/hari ke 2-4 langganan - praktis tak terbatas
Repositori kode	GitHub - dua repositori (publik/privat bebas)	repo pribadi/publik gratis tanpa batas jumlah	2 repo, push jarang
Sinkronisasi waktu	Server NTP publik (pool.ntp.org)	publik gratis tanpa akun	1 sinkronisasi per boot
Root CA TLS	pki.goog (GTS Root R1/R4)	sertifikat root publik gratis	tertanam di firmware - tanpa panggilan runtime

Tabel 10.1 - Peta platform gratis, batas kuotanya, dan beban nyata instalasi maksimum.

Dua catatan penting dari tabel tersebut. Pertama, satu-satunya biaya nyata yang tersisa adalah koneksi internet lokasi (WiFi langganan/pulsa yang sudah Anda bayarkan sebelum sistem ini ada) - sistem menambahkan beban traffic yang bisa diabaikan. Kedua, karena tidak ada satu pun kartu kredit yang terpasang di seluruh rantai, secara teknis TIDAK MUNGKIN muncul tagihan tak terduga; skenario terburuk saat batas gratis tersentuh adalah pembatasan sementara (lihat 10.3), bukan penagihan.

10.2 MQTT: tidak dibutuhkan - dan itu sengaja

Sistem ini sama sekali TIDAK memakai MQTT atau broker pesan apa pun. Firmware cukup melakukan satu HTTPS POST per menit langsung ke endpoint GAS, GAS menyimpan status terakhir dan mengirim push saat tepi alarm, dan klien membaca snapshot dengan polling 30 detik saat aplikasi terbuka. Arsitektur tanpa broker ini adalah keputusan desain yang menghilangkan satu platform berbayar penuh dari kebutuhan: tidak ada server MQTT yang harus disewa, diamankan, dan dijaga tetap hidup. Untuk kebutuhan dua HP dan satu modul, latensi maksimum satu menit ini tidak terasa; alarm tetap tampil walau aplikasi tertutup karena jalurnya adalah Web Push, bukan MQTT.

Bila suatu saat kebutuhan berubah (misalnya puluhan modul, kontrol aktuator sub-detik, atau dasbor real-time tanpa polling), berikut opsi broker MQTT yang punya jalur gratis - lengkap dengan kehati-hatian pemakaiannya:

- **EMQX Cloud Serverless:** tier gratis dengan kuota koneksi bulanan; daftar akun saja. Verifikasi ketentuan terkini di situs resmi sebelum dipakai (syarat tier gratis dapat berubah).
- **HiveMQ Cloud:** paket gratis untuk jumlah perangkat kecil; cocok untuk armada belasan perangkat.
- **test.mosquitto.org:** broker publik TANPA autentikasi - hanya untuk eksperimen belajar; JANGAN dipakai produksi (data terbuka siapa pun).
- **Mosquitto self-host:** broker gratis total di Raspberry Pi/komputer mini di lokasi - biaya hanya perangkat dan listrik; butuh jaringan yang bisa menerima koneksi masuk (port forwarding atau VPN).

Penting dipahami: memakai MQTT menuntut perubahan arsitektur nyata (firmware baru, konsumen broker, penanganan kredensial broker, dan tetap perlu jalur Web Push untuk notifikasi saat aplikasi tertutup). Untuk instalasi ini, jalur HTTPS-ke-GAS yang sedang berjalan sudah memenuhi seluruh kebutuhan dengan biaya nol - tidak ada broker yang perlu ditambahkan sekarang.

10.3 Batas yang benar-benar relevan dan perilakunya

- **Kuota GAS habis:** reset otomatis TIAP HARI (waktu pasifik). Bila tersentuh, laporan firmware ditolak sementara - firmware merespons dengan backoff eksponensial dan melanjutkan sendiri setelah reset; tidak ada data yang rusak. Dengan beban 2 HP + 1 modul, kuota 20.000 panggilan keluar/hari praktis tidak tersentuh (pemakaian normal di bawah 100/hari).
- **Vercel Hobby:** batas ToS-nya adalah penggunaan NON-KOMERSIAL. Untuk pemantauan pribadi/keluarga - selamanya gratis. Bila kelak dipakai komersial, jalur tetap-gratis tersedia: pindahkan hosting ke Cloudflare Pages (bandwidth statis gratis tak terbatas) - lihat 10.4.
- **TTL push 24 jam:** pesan alarm untuk HP yang offline lebih dari 24 jam tidak lagi terkirim (push service membuangnya). Untuk alarm monitoring ini perilaku yang benar - alarm lama tidak boleh tumpang tindih alarm baru.
- **iOS:** Web Push hanya aktif untuk PWA yang dipasang ke Layar Utama (Add to Home Screen, iOS 16.4+) - gratis, tanpa App Store, tanpa akun developer Apple.
- **Pemantauan pemakaian (semua gratis):** dashboard Vercel - tab Usage/Limits; editor GAS - Executions. Tidak ada penagihan yang mungkin muncul mendadak karena tidak ada kartu terpasang.

10.4 Kompatibilitas jalur alternatif yang tetap gratis

Sistem dirancang agar tidak terkunci pada satu penyedia. PWA standalone adalah berkas statis murni - kompatibel dengan semua hosting statis gratis: Netlify Drop (tanpa akun pun bisa), Cloudflare Pages, GitHub Pages, Firebase Hosting. Aplikasi Next.js utama paling native di Vercel, tetapi tetap punya jalur gratis bila suatu hari dipindahkan: Netlify (adapter Next.js resmi) atau Cloudflare Pages (opennext-cloudflare). Persyaratan teknis yang harus dipertahankan di penyedia mana pun hanya dua: HTTPS aktif di seluruh situs,

dan `sw.js` disajikan dari root scope dengan header `Cache-Control no-cache` (`vercel.json` pada repo adalah contoh konfigurasinya untuk Vercel; penyedia lain punya berkas konfigurasi setara - semuanya didokumentasikan penyediaanya masing-masing).

Backend GAS tidak memiliki dependensi hosting: seluruh kebutuhannya (Web App, Properties, `UrlFetch`) adalah bagian dari akun Google gratis. Firmware juga tidak terkunci - selama jaringan 2,4 GHz dapat mencapai internet dan DNS dapat menyelesaikan `script.google.com`, modul bekerja di belakang router lokasi mana pun. Dengan demikian, biaya nol bukan kebetulan sesaat: setiap lapis sistem dipilih karena jalur gratisnya yang stabil dan cukup, dan tiap lapis punya jalur alternatif gratis bila ketentuan penyedia berubah di masa depan.